Dauphine | PSL

ds_finder

L3 MIAGE APP

Projet en algorithmes dans les graphes (PACE 2025)

Hamza Konte
Enseignant : Florian Sikora
LAMSADE - Université Paris-Dauphine PSL
Février 2026

1 Introduction

Le *Minimum Dominating Set* (MDS) est un problème d'optimisation classique sur les graphes : étant donné un graphe non orienté $G = (V, E)$, on cherche un sous-ensemble $D \subseteq V$ de taille minimale tel que tout sommet de V est soit dans D, soit adjacent à un sommet de D[1]. Ce problème est NP-difficile dans le cas général et constitue le thème du challenge PACE 2025[2].

Ce solveur a été écrit en C (≈ 800 lignes) qui combine des heuristiques gloutonnes adaptatives et une recherche locale de type tabou pour produire des solutions de bonne qualité en temps limité. Le programme reçoit un graphe au format DIMACS sur l'entrée standard, résout le problème sous contrainte de temps (SIGTERM), et affiche la solution sur la sortie standard.

2 Choix techniques

2.1 Choix du langage

Le C n'est pas un choix magique : un développeur Python expérimenté peut aisément surpasser un débutant en C sur un problème donné. L'intérêt du C réside dans le **contrôle** qu'il offre : gestion explicite de la mémoire, maîtrise du layout des données, absence de couche d'abstraction entre le code et la machine.

Le revers de la médaille est un besoin permanent de **débogage** : fuites mémoire, segfaults, dépassements de buffer. Ce coût est réel et significatif. Deux pratiques se sont révélées indispensables pour y faire face :

- **L'intégration continue (CI)** : chaque push déclenche une batterie de tests incluant Valgrind, ce qui détecte automatiquement les régressions (fuites mémoire, solutions invalides, crashes).
- **Committer régulièrement** : des commits fréquents et atomiques permettent d'isoler rapidement le changement responsable d'un bug. Un commit de 500 lignes est un cauchemar à déboguer ; un commit de 20 lignes permet de localiser le problème en quelques minutes.

Ces deux pratiques forment un filet de sécurité qui rend le C viable sur un projet de cette envergure[3].

2.2 Structures de données minimalistes

Le premier réflexe pour représenter un graphe est la **matrice d'adjacence**, c'est-à-dire un tableau $V \times V$ de booléens. Ce choix a été le point de départ du projet, mais le programme crashait sur les grosses instances : une matrice pour $V = 100000$ sommets occupe ≈ 10 Go, ce qui dépasse la mémoire disponible.

Le pivot vers les **listes d'adjacence** ($O(V + E)$ en mémoire) a résolu le problème. Au-delà de ce choix fondamental, l'ensemble des structures du solveur suit une philosophie minimaliste :

- Tableaux bruts `int[]`, pas de hash maps ni de structures complexes.
- `t_bool` = `unsigned char`, soit un octet par booléen, au lieu de 4 avec `int`.
- Réutilisation de buffers pré-alloués plutôt que des allocations dynamiques dans les boucles chaudes.
- Accès mémoire séquentiels pour maximiser l'utilisation du cache CPU (*cache-friendly*).

Le minimum nécessaire, pas de superflu : chaque structure est dimensionnée exactement à V ou $V + E$, sans marge ni indirection.

2.3 Importance de l'optimisation

Le challenge PACE impose un timeout strict : le solveur reçoit un SIGTERM et doit avoir produit sa meilleure solution. Dans ce contexte, la performance est la **contrainte fondamentale** : plus le code est rapide, plus l'optimiseur tabou effectue d'itérations, et meilleure est la solution avant l'arrêt.

L'optimisation va **au-delà de l'algorithme**. Un même algorithme peut s'exécuter jusqu'à 2× plus vite avec une attention portée à la compilation et au layout mémoire : flags `-O3 -flto -march=native`, accès cache séquentiels, évitement des allocations dans les boucles chaudes. Il ne suffit pas de faire un code qui marche, il faut comprendre ce qu'il se passe *derrière* à la compilation.

Ce type d'optimisation bas-niveau est un domaine où l'IA actuelle atteint ses limites : elle peut suggérer des algorithmes, mais le profiling empirique, le paramétrage fin et l'optimisation de performance restent du ressort de l'humain.

Travailler sur un problème NP-difficile sous contrainte de temps a enseigné des notions qui vont au-delà du cours : rigueur de développement, pensée systémique, importance du profiling empirique face à l'intuition.

---

[1] Pour une définition formelle et la preuve de NP-difficulté, voir Sikora, F., *Notes de cours : Algorithmes dans les graphes*, Université Paris-Dauphine, 2025–2026. Voir aussi Haynes, T. W., Hedetniemi, S. T. & Slater, P. J. (1998), *Fundamentals of Domination in Graphs*, Marcel Dekker.

[2] *Parameterized Algorithms and Computational Experiments*, https://pacechallenge.org/2025/

[3] Description du challenge PACE 2025, *Minimum Dominating Set*, *Parameterized Algorithms and Computational Experiments*. Voir https://pacechallenge.org/2025/ pour les contraintes de soumission (format DIMACS, timeout SIGTERM, binaire x86_64).

3 Architecture et pipeline

Le solveur suit un pipeline en cinq étapes :

Étape	Description
1. Lecture	Lecture du graphe au format DIMACS (`p ds V E` puis arêtes). Construction des listes d'adjacence dynamiques (doublement de capacité, complexité amortie $O(V+E)$).
2. Classification	Heuristiques empiriques pour détecter la topologie du graphe (étoile, arbre, grille, core-périphérie…). Permet de choisir le solveur adapté.
3. Tri	Tri des sommets par degré croissant (`qsort`, $O(V \log V)$). Le glouton sélectionnera les sommets de plus haut degré en priorité.
4. Glouton	Construction d'une solution initiale par heuristique gloutonne. Deux variantes selon le type de graphe (voir §3.1).
5. Optimiseur	Recherche locale tabou : suppression de sommets redondants (*prune*), remplacement intelligent (*swap*), perturbation contrôlée pour échapper aux optima locaux.

Tableau 1. – Pipeline de résolution

3.1 Schéma du workflow

Le diagramme ci-dessous résume le flux de données à travers le pipeline, de la lecture de l'entrée jusqu'à l'écriture de la solution :

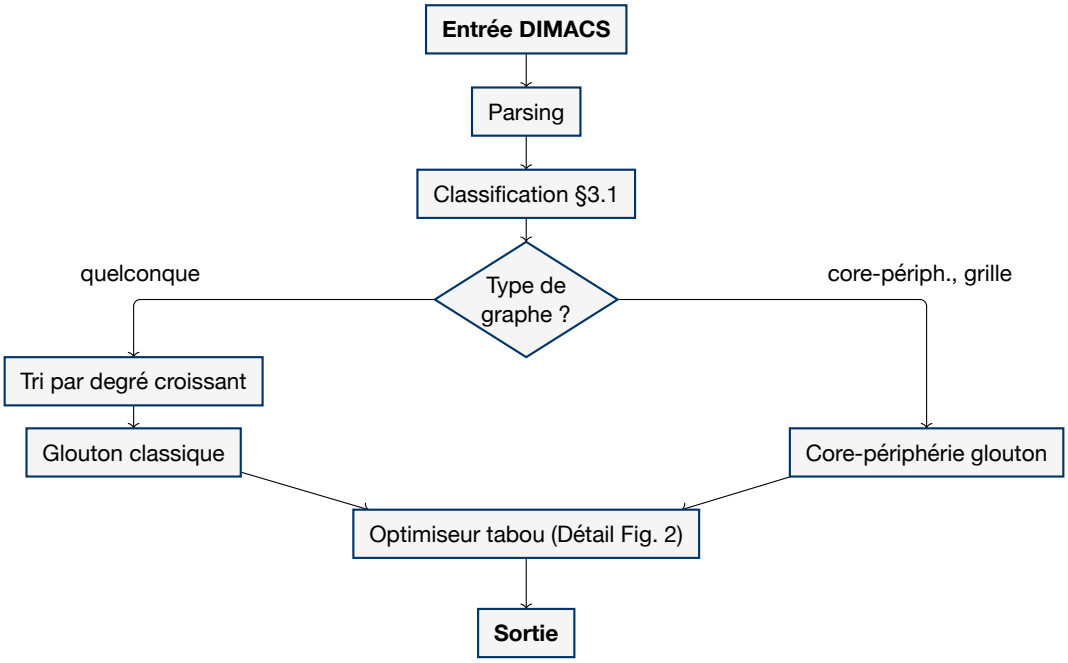

Fig. 1. – Workflow global du pipeline de résolution

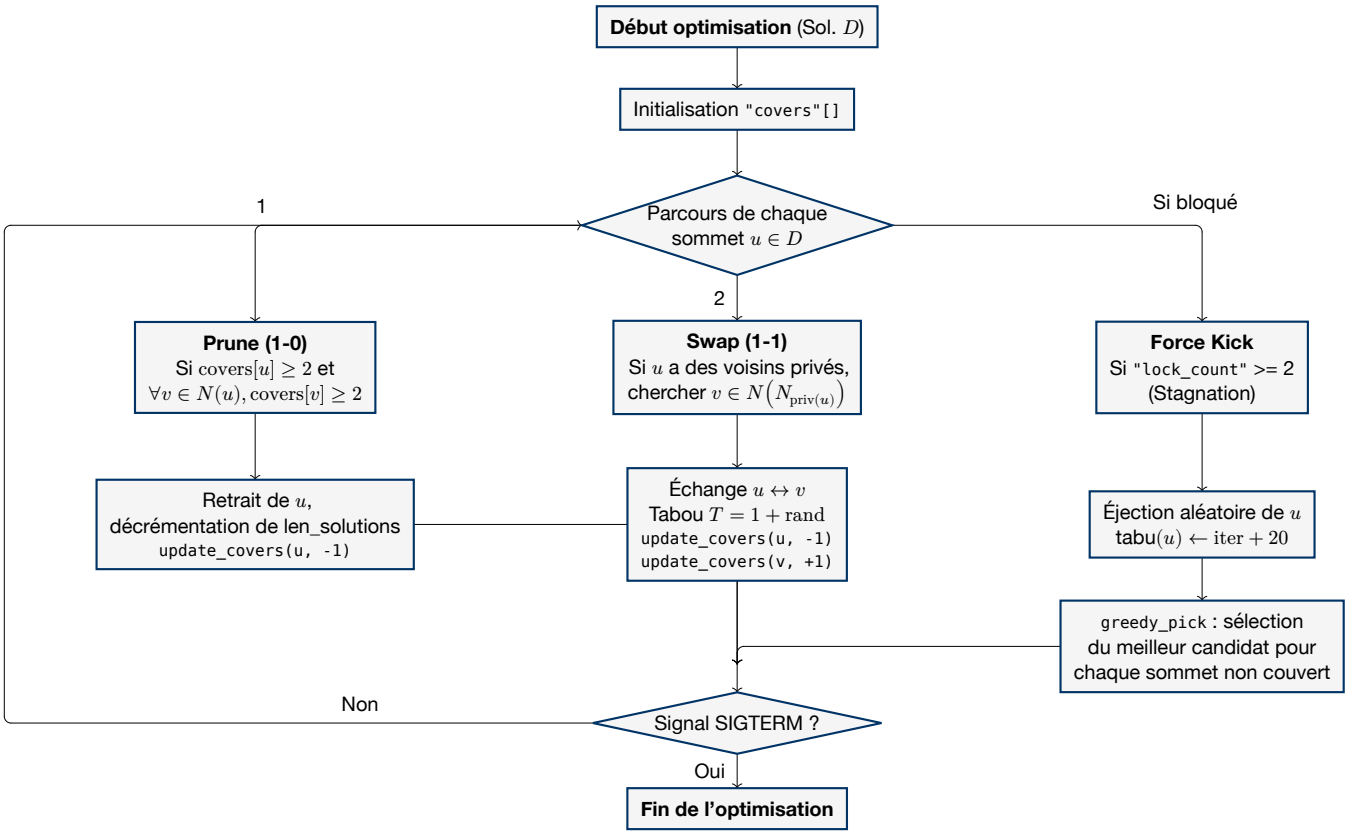

Fig. 2. – Workflow de l'optimiseur par recherche locale et perturbation

4 Algorithmes et complexités

4.1 Classification de graphe

La fonction `get_graph_type` calcule le degré maximal ($d_{\max}$), le degré moyen ($\overline{d}$), le ratio $r = d_{\max} \,/\, \overline{d}$, et le nombre de feuilles. Ces métriques permettent de classifier le graphe parmi dix types[4] :

Type	Condition	Solveur
Complet	$\lvert E\rvert = \frac{V(V-1)}{2}$	Glouton classique
Étoile	$d_{\max} = V - 1$, $\lvert E\rvert = V - 1$	Glouton classique
Multi-étoile	$r > 50$, feuilles $> 70\%$	Glouton classique
Core-périphérie	$r > 50$, feuilles $\leq 70\%$	Core-périphérie glouton
Scale-free	$r > 5$	Glouton classique
Arbre	$\lvert E\rvert = V - 1$, $\overline{d} \in [2, 6]$	Glouton classique
Anneau	$\lvert E\rvert = V$, $\overline{d} \in [2, 6]$	Glouton classique
Grille 2D	$\overline{d} \in [2, 6]$, $d_{\max} \leq 10$	Core-périphérie glouton
Erdős–Rényi	feuilles $= 0$, $\overline{d} > 2$	Glouton classique
Inconnu	Défaut	Glouton classique

Tableau 2. – Heuristiques de classification

Complexité : $O(V)$, un seul parcours des sommets suffit.

4.2 Glouton classique

L'algorithme glouton par degré maximal[5] procède en deux phases :

1. **Phase feuilles** : pour chaque sommet de degré 1 encore actif (non dominé), son unique voisin est ajouté à D. Ce choix est localement optimal : le voisin domine la feuille et potentiellement d'autres sommets adjacents, alors que la feuille elle-même ne couvrirait qu'elle-même et son voisin.

[4] L'idée de classifier les instances par topologie pour adapter la stratégie de résolution s'inspire des approches décrites dans plusieurs solveurs PACE, notamment da Fonseca, G. D., Feschet, F. & Gerard, Y., *Shadoks Approach to Minimum Hitting Set and Dominating Set*, PACE 2025 ; et Geis, L. et al., *PaceYourself: Heuristic and Exact Solvers for the Minimum Dominating Set Problem*, PACE 2025.

[5] L'approche gloutonne par degré pour le *Minimum Dominating Set* est présentée dans Fontan, F. & Verger, G., *Solver description*, PACE 2025 ; et Swat, S., *DS7HS*, PACE 2025.

2. **Phase principale** : le critère de sélection est le **degré** $\deg(u)$. À chaque itération, le sommet actif de plus haut degré est ajouté à D. L'intuition est qu'un sommet de haut degré domine davantage de voisins par ajout, ce qui tend à minimiser $|D|$. Ses voisins sont ensuite marqués comme dominés (inactifs).

Le tableau `v_sorted` (trié par degré croissant) est parcouru en sens inverse pour accéder au sommet de degré maximal en $O(1)$ amorti. Un pointeur `first_active` mémorise l'index du premier sommet encore actif pour éviter de re-parcourir les sommets déjà inactifs.

Complexité : $O(V + E)$, chaque arête est visitée au plus deux fois (sélection + mise à jour).

4.3 Core-périphérie glouton

Variante basée sur le **gain**[6]. Contrairement au glouton classique qui utilise le degré statique, le critère ici est le **gain dynamique** :

$$\text{gain}(u) = \mathbf{1}_{\text{actives}[u]} + |\{v \in N(u) : \text{actives}[v]\}|$$

C'est-à-dire le nombre de sommets actuellement non dominés que u couvrirait s'il était ajouté à D (lui-même inclus s'il est encore actif, plus ses voisins actifs). Ce gain évolue au fil des ajouts : quand un sommet v est ajouté à D, les voisins de v deviennent inactifs, ce qui réduit le gain des sommets environnants.

L'algorithme utilise un *bucket sort* par gain pour éviter un tri complet à chaque itération :

1. **Initialisation** : chaque sommet est placé dans le bucket correspondant à son degré $+1$. Cette valeur est une surestimation du gain réel car, au départ, tous les sommets sont actifs et $\text{gain}(u) = \deg(u) + 1$. Au fur et à mesure que des sommets sont dominés, le gain réel diminue. Structure : tableaux `head[g]`, `next[u]`, `gain[u]` formant des listes chaînées implicites.
2. **Sélection** : le sommet u est extrait du bucket de gain le plus élevé. Son gain réel est recalculé à ce moment (*lazy evaluation*) :
   - Si le gain réel $\geq$ le gain du bucket : le gain n'a pas diminué (ou a été sous-estimé), u reste un bon candidat et est ajouté à D.
   - Sinon : le gain a diminué depuis le placement initial (des voisins ont été dominés entre-temps), u est replacé dans le bucket correspondant à son gain réel (*re-bucketing*).

Cette stratégie est particulièrement efficace pour les graphes à structure core-périphérie et les grilles, où les gains évoluent de façon non uniforme.

Complexité : $O(V^2 + E)$ dans le pire cas (re-bucketing répété), $O(V \log V)$ en pratique.

4.4 Optimiseur : recherche tabou

L'optimiseur itère sur la solution courante et tente d'en réduire la taille par trois opérations[7] :

4.4.1 Prune (suppression)

Le critère de prune repose sur le tableau `covers[]`, où $\text{covers}[v]$ compte le nombre de dominateurs de v dans la solution courante : v lui-même s'il est dans D, plus chaque voisin de v présent dans D. Un sommet est couvert si et seulement si $\text{covers}[v] \geq 1$.

Un sommet $u \in D$ est retiré si $\text{covers}[u] \geq 2$ **et** $\forall v \in N(u), \text{covers}[v] \geq 2$[8]. Le seuil 2 garantit qu'après le retrait de u (qui décrémente de 1 les compteurs de u et de tous ses voisins), chaque sommet affecté conserve $\text{covers} \geq 1$ et reste donc dominé. La mise à jour est incrémentale en $O(\deg(u))$.

4.4.2 Swap (remplacement)

On identifie les *voisins privés* de u : les sommets pour lesquels $\text{covers}[v] = 1$, c'est-à-dire les sommets couverts **uniquement** par u. Ce sont les seuls sommets qui deviendraient non couverts si u était retiré de D — tous les autres voisins ont au moins un autre dominateur.

Le critère de swap est le suivant : on cherche un sommet $w \notin D$ tel que w est adjacent à **tous** les voisins privés de u (et w lui-même couvre ceux qui lui sont identiques). Si un tel w existe, il peut « reprendre le rôle » de u : on remplace u par w dans D sans créer de sommet non couvert, et

$|D|$ reste inchangé[9]. L'espace de recherche est restreint aux voisins du premier voisin privé ($N(v_0)$), car w doit au minimum être adjacent à v_0.

Une **liste tabou** empêche les cycles : après un swap, u et w sont interdits pendant $T = 1 + \text{rand}(0..9)$ itérations. La composante aléatoire de la tenure évite les boucles déterministes où les mêmes paires seraient échangées indéfiniment.

Si le nombre de voisins privés dépasse 15, le swap est abandonné immédiatement : avec autant de contraintes, la probabilité qu'un seul sommet w soit adjacent à tous les voisins privés est très faible (surtout sur les graphes creux), et le coût de vérification $O(15 \times \deg(w))$ serait gaspillé.

Complexité par swap : $O(\deg(u)^2)$ dans le pire cas, mais l'abandon précoce borne le coût en pratique.

4.4.3 Perturbation (force kick)

Quand aucune amélioration n'est trouvée pendant 2 itérations consécutives, une perturbation est déclenchée[10] :
1. Un sommet aléatoire u est retiré de D (avec tabu de 20 itérations pour empêcher sa réinsertion immédiate).
2. Les sommets devenus non couverts sont réparés par la fonction `greedy_pick`, détaillée ci-dessous.

4.4.3.1 Sélection gloutonne : `greedy_pick`

Au cœur de la phase de réparation, `greedy_pick(g, covers, solutions, u, include_self)` sélectionne le meilleur candidat à ajouter à D parmi le voisinage d'un sommet non couvert u. L'algorithme procède en deux passes :

1. **Évaluation de u** (si `include_self` et $u \notin D$) : le score de u est le nombre de ses voisins actuellement non couverts ($\text{covers}[v] = 0$).

---

[6] Les techniques de bucket sort par gain et de lazy evaluation sont utilisées dans plusieurs solveurs PACE pour accélérer la construction gloutonne. Voir notamment Luo, C. et al., *Weighting-based Local Search Heuristic for the Hitting Set Problem*, PACE 2025 ; et Chhabra, A. et al., *LetsJustCHILS*, PACE 2025.

[7] Les principes de recherche locale tabou appliqués au MDS sont décrits en détail dans Zhu, E. et al. (2024), *A dual-mode local search algorithm for solving the minimum dominating set problem*, Knowledge-Based Systems 298 ; et Cai, S. et al. (2020), *Two-goal Local Search and Inference Rules for Minimum Dominating Set*, IJCAI-20.

[8] La suppression de sommets redondants est une technique centrale dans Zhu, E. et al. (2024), *A dual-mode local search algorithm for solving the minimum dominating set problem*, où elle constitue le premier mode.

[9] Le mécanisme de swap basé sur les voisins privés est inspiré de Cai, S. et al. (2020), *Two-goal Local Search and Inference Rules for Minimum Dominating Set*, IJCAI-20, où le remplacement 1-1 constitue l'un des deux objectifs de la recherche locale.

[10] Les mécanismes de perturbation pour échapper aux optima locaux sont étudiés dans Cai, S. et al. (2020), *Two-goal Local Search*, IJCAI-20, et dans Zhu, E. et al. (2024), *A dual-mode local search*, Knowledge-Based Systems. Voir aussi les stratégies de perturbation dans da Fonseca, G. D. et al., *Shadoks*, PACE 2025.

2. **Évaluation de chaque voisin $v \in N(u)$** (si $v \notin D$) : le score de v est $\mathbf{1}_{\text{covers}[v]=0} + |\{w \in N(v) : \text{covers}[w] = 0\}|$, c'est-à-dire le nombre total de sommets non couverts que v dominerait s'il était ajouté à D (lui-même inclus s'il est non couvert).

Le candidat de score maximal est retourné. Si aucun candidat valide n'existe, la fonction retourne -1.

Complexité : $O\big(\deg(u) \times \overline{d}\big)$ — une passe sur $N(u)$, et pour chaque voisin v, une passe sur $N(v)$.

4.4.3.2 Orchestration : `add_candidates`

La procédure `add_candidates` utilise `greedy_pick` en deux phases après l'éjection du sommet u :

1. **Phase 1 — réparation des voisins :** pour chaque voisin v_i de u devenu non couvert ($\text{covers}[v_i] = 0$), `greedy_pick` est appelé avec `include_self = TRUE`. Le meilleur candidat dans $N[v_i]$ (potentiellement v_i lui-même) est ajouté à D, et `covers[]` est mis à jour immédiatement. Cette mise à jour **dynamique** est cruciale : chaque ajout modifie les scores des candidats suivants, ce qui évite d'ajouter des sommets redondants.
2. **Phase 2 — réparation du sommet éjecté :** si u lui-même reste non couvert après la phase 1, `greedy_pick` est appelé avec `include_self = FALSE` pour sélectionner le meilleur voisin de u — sans considérer u lui-même comme candidat, car il est interdit par le tabou.

Comparée à une réparation naïve (ajout du premier voisin non couvert), cette sélection gloutonne dynamique réduit la taille de la solution d'environ 83 sommets sur les instances larges à 300 secondes de calcul.

Trois mécanismes complètent la perturbation :

- **Restauration retardée** : si la solution courante se dégrade, c'est-à-dire si $|D_{\text{courante}}| > |D_{\text{meilleure}}|$ (la taille de la solution a augmenté par rapport à la meilleure connue), le solveur tolère 2 itérations de dégradation avant de restaurer l'état sauvegardé. Cela laisse à la recherche locale le temps de traverser un col et potentiellement trouver une meilleure vallée de l'autre côté.
- **Perturbation croissante** : après 20 perturbations consécutives sans amélioration globale (c'est-à-dire sans que $|D_{\text{meilleure}}|$ ne diminue), le solveur déclenche une triple perturbation (3 éjections successives) pour diversifier plus agressivement l'exploration et sortir d'un bassin d'attraction.
- **Générateur XOR-shift** déterministe (graine fixe 42) pour assurer la reproductibilité[11].

Complexité globale de l'optimiseur : $O\big(I \times V \times \overline{d}\big)$ où I est le nombre d'itérations (borné par le temps).

Composant	Temps	Espace
Lecture + construction	$O(V + E)$	$O(V + E)$
Classification	$O(V)$	$O(1)$
Tri	$O(V \log V)$	$O(V)$
Glouton classique	$O(V + E)$	$O(V)$
Core-périphérie glouton	$O(V^2 + E)$ pire cas	$O(V)$
Prune	$O(\deg(u))$	$O(V)$
Swap	$O(\deg(u)^2)$	$O(V)$
Perturbation (`greedy_pick`)	$O\big(\deg(u) \times \overline{d}\big)$	$O(1)$

Tableau 3. – Récapitulatif des complexités

5 Chronologie du développement

Le projet s'est développé sur trois mois, de début décembre 2025 à début février 2026. L'historique Git compte plus de 130 commits, synthétisés ci-dessous.

Période	Phase de développement
7–8 déc.	**Fondations.** Parseur DIMACS, structure de graphe (listes d'adjacence), Docker, premier solveur, arêtes bidirectionnelles.
9–14 déc.	**Kernelization et glouton.** Réduction de feuilles, glouton par degré max. CI GitHub.
23–24 déc.	**Scoring et optimiseur.** Sélection par score, premier optimiseur prune-swap tabou. Migration CMake. Timeout SIGTERM.
27–28 déc.	**Stabilisation.** Correction de fuites mémoire, refactoring des types (`int` unifié), timeout robuste, `printf` bufférisé, CI timeout.
31 déc. – 3 jan.	**Core-périphérie glouton.** Bucket sort par gain, lazy evaluation. Classification de graphe. Tri adaptatif selon le type.
4–8 jan.	**Optimiseur avancé.** Force kick (perturbation), détection de stagnation, rollback. Refactoring et nettoyage.
10–11 jan.	**Affinement.** Rollback conditionnel, allocation mémoire optimisée.
31 jan. – 8 fév.	**Finalisation.** Réparation gloutonne, tenure réduite, restauration retardée, perturbation croissante, rapport.

Tableau 4. – Chronologie synthétique du développement

6 Méthode de travail

6.1 Intégration continue (CI GitHub)

Le développement s'appuie sur un pipeline d'intégration continue défini dans `.github/workflows/dockerized.yml`. Cinq jobs s'exécutent en parallèle à chaque push :

[11] Marsaglia, G. (2003). Xorshift RNGs. *Journal of Statistical Software*, 8(14), 1–6.

Job	Rôle
build	Compilation statique x86_64 via Docker, produit le binaire soumis à Optil.io.
compile	Vérification de compilation en mode Release (`-O3`, `-flto`), détecte les régressions de build.
test	Exécution de `testall.sh` sur le jeu de test avec le vérificateur Java `ds_verifier-1.0.jar`. Timeout de 60s par instance. Détection des statuts TO (timeout), KO (solution invalide), ERR (erreur).
leaks	Valgrind (`--leak-check=full`, `--error-exitcode=42`) sur une instance de test, garantit l'absence de fuites mémoire.
timeout	Envoi de SIGTERM après 15s et vérification que la solution partielle reste valide, simule le comportement d'Optil.io.

Tableau 5. – Jobs du pipeline CI

Cette CI a permis de détecter plusieurs régressions silencieuses, notamment des fuites mémoire introduites lors de refactorings et un cas où la solution partielle après SIGTERM était corrompue.

6.2 Docker et cross-compilation

Le développement s'est fait sur Apple Silicon (ARM), alors que la plateforme de soumission Optil.io exige un binaire ELF x86_64. Les premières soumissions étaient systématiquement rejetées : le binaire ARM n'était tout simplement pas exécutable sur la machine cible.

La solution a été mise en place via `docker-compose.yml` avec deux services :

- **ds-dev** : conteneur de développement embarquant `gcc`, `valgrind` et `java` (pour le vérificateur). Volumes montés sur `src/`, `testset/` et `real-instances/` pour itérer sans reconstruire l'image.
- **ds-build** : conteneur de build avec `platform: linux/amd64`, qui produit un binaire ELF x86_64 statiquement lié via CMake. C'est ce binaire qui est soumis à Optil.io.

Ce setup Docker a débloqué les soumissions et a aussi rendu la CI GitHub possible (même image de build en local et sur les runners).

6.3 Méthode A/B testing

Chaque modification algorithmique a été validée par une comparaison directe avant/après sur les instances du benchmark PACE, via le script `testall.sh`. Le script exécute le solveur sur chaque instance `.gr` avec un timeout configurable, vérifie la validité de la solution avec le vérificateur Java, et produit un fichier `results.csv` avec la taille de la solution (ou le statut d'erreur) par instance.

Cette approche a permis de quantifier l'impact de chaque changement. Par exemple, le constat du 30 décembre (glouton naïf insuffisant sur les petites instances) a été confirmé objectivement par les résultats du professeur publiés le 3 janvier, qui montraient que le nouveau glouton trié surpassait systématiquement l'ancienne version.

6.4 Visualisation des solutions

Un outil de visualisation en Python (NetworkX + Matplotlib) a été développé en parallèle du solveur, accompagné d'un convertisseur GEXF pour Gephi. Ces outils permettent de représenter graphiquement les graphes et les solutions calculées par le solveur.

La visualisation a joué un rôle important dans le processus itératif d'amélioration :

- **Comprendre les choix du solveur** : en colorant les sommets sélectionnés par l'heuristique, on observe immédiatement pourquoi certains nœuds sont choisis (hubs dans les graphes scale-free, connecteurs dans les grilles).
- **Identifier les faiblesses** : des sommets redondants (dominateurs inutiles) deviennent visibles à l'œil, ce que les seuls chiffres du benchmark ne révèlent pas.
- **Valider la classification** : vérifier visuellement qu'un graphe classé comme « grille » ou « core-périphérie » correspond bien à la topologie attendue.
- **Guider les optimisations** : l'observation que le glouton naïf ne sélectionnait pas assez de sommets en périphérie a directement conduit à l'idée du *periphery greedy*. Le coût de dominer le cœur en premier était plus élevé que de commencer par la périphérie, un constat difficile à faire sans représentation visuelle.

Cet outil de développement ne fait pas partie du solveur soumis, mais il a été essentiel au processus de conception et de débogage.

7 Utilisation de l'IA générative

L'outil **Claude Code** (CLI d'Anthropic, modèle Claude Opus 4.5) a été utilisé ponctuellement au cours du développement. Voici les tâches pour lesquelles l'IA a été sollicitée, ainsi qu'une appréciation de son apport :

- **Débogage et diagnostic** : aide à la compréhension de comportements inattendus : fuites mémoire détectées par Valgrind (notamment sur le parseur `get_next_line`), conditions de course entre le timeout SIGTERM et l'écriture de la solution. L'IA a été utile pour formuler des hypothèses et orienter la recherche, mais la correction effective a nécessité une compréhension fine du code.
- **Exploration algorithmique** : discussion sur les stratégies de perturbation (force kick vs. restart complet), sur le choix des paramètres de la recherche tabou (tenure, seuil de stagnation), et sur les critères de classification de graphe. L'IA a proposé des pistes intéressantes, mais les décisions finales ont été prises après expérimentation sur les instances du benchmark PACE.
- **Génération de commentaires** : ajout de commentaires explicatifs sur le code existant pour le rendu final. Tâche purement mécanique où l'IA a été très efficace.

Retour critique : L'IA s'est avérée utile pour les tâches mécaniques (déboggage, commentaires, mise en forme) et pour accélérer l'exploration de pistes algorithmiques. Cependant, les choix d'architecture fondamentaux (classification de graphe, stratégie d'optimisation, paramétrage de la recherche tabou) ont été conçus et ajustés manuellement par itérations successives sur les instances de test. Le paramétrage fin (tenure tabou de 1, seuil de perturbation à 2 itérations, tabu de 20 pour le force kick, seuil d'abandon précoce du swap à 15) résulte d'une expérimentation empirique que l'IA ne peut pas remplacer. En résumé, l'IA réduit significativement le temps passé sur les tâches à faible valeur ajoutée, mais ne se substitue pas à la compréhension du problème ni au travail d'expérimentation.

8 Références

1. Sikora, F. *Notes de cours : Algorithmes dans les graphes*. L3 MIAGE Apprentissage, Université Paris-Dauphine, 2025–2026.
2. PACE 2025 Challenge, Minimum Dominating Set. *Parameterized Algorithms and Computational Experiments*. https://pacechallenge.org/2025/
3. Haynes, T. W., Hedetniemi, S. T., & Slater, P. J. (1998). *Fundamentals of Domination in Graphs*. Marcel Dekker.
4. Glover, F. & Laguna, M. (1997). *Tabu Search*. Kluwer Academic Publishers.
5. Grandoni, F. (2006). A note on the complexity of minimum dominating set. *Journal of Discrete Algorithms*, 4(2), 209–214.
6. Zhu, E., Zhang, Y., Wang, S., Strash, D., & Liu, C. (2024). A dual-mode local search algorithm for solving the minimum dominating set problem. *Knowledge-Based Systems*, 298, 111950.
7. Cai, S., Hou, W., Wang, Y., Luo, C., & Lin, Q. (2020). Two-goal Local Search and Inference Rules for Minimum Dominating Set. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*.
8. da Fonseca, G. D., Feschet, F., & Gerard, Y. Shadoks Approach to Minimum Hitting Set and Dominating Set. PACE 2025 Solver Description.
9. Swat, S. DS7HS: Dominating Set and Hitting Set Solver. PACE 2025 Solver Description, Poznań University of Technology.
10. Luo, C., Zhang, Q., & Su, Z. Weighting-based Local Search Heuristic for the Hitting Set Problem. PACE 2025 Solver Description, Huazhong University of Science and Technology.
11. Chhabra, A., Dittes, M., Großmann, E., Langedal, K., & Reinstädtler, H. LetsJustCHILS. PACE 2025 Solver Description, Heidelberg University.
12. Geis, L., Leonhardt, A., Meintrup, J., Meyer, U., & Penschuck, M. PaceYourself: Heuristic and Exact Solvers for the Minimum Dominating Set Problem. PACE 2025 Solver Description, Goethe University Frankfurt.
13. Fontan, F. & Verger, G. Solver description. PACE 2025 Solver Description.
14. Marsaglia, G. (2003). Xorshift RNGs. *Journal of Statistical Software*, 8(14), 1–6.
15. Wan, P.-J., Alzoubi, K. M., & Frieder, O. (2004). Distributed construction of connected dominating set in wireless ad hoc networks. *Mobile Networks and Applications*, 9(2), 141–149.
16. Optil.io, Online platform for optimization challenges. https://optil.io/